

Linux для научной работы

Терминал, SSH, tmux, редакторы и Git

Саватейкин Ярослав Михайлович

Химический факультет МГУ

2026

Linux и терминал

Зачем Linux в научной работе

На ноутбуке

Пишем код, читаем результаты, работаем с графическим интерфейсом.

Терминал — не «чёрный экран для программистов», а точный способ давать машине команды и воспроизводить работу.

На сервере или кластере

Запускаем расчёты, храним данные, используем вычислительные ресурсы.

Базовая модель Linux

Ядро

Распределяет ресурсы
компьютера.

Программы

Python, редактор, браузер.

Пользователь

Управляет системой через
GUI или shell.

Linux — семейство операционных систем. **Дистрибутив** собирает ядро, программы и настройки; Ubuntu — один из популярных примеров.

Окно терминала и shell

Окно терминала

Окно, в котором виден текст.



Shell

Читает команды и запускает программы.

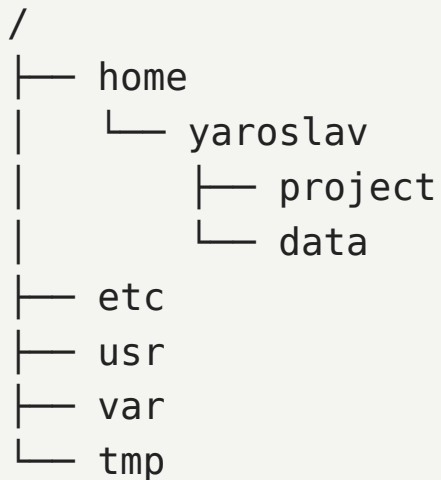
На сервере GUI часто нет: это экономит ресурсы и упрощает удалённую работу.

Как читать командную строку

```
yaroslav@laptop:~/project$ pwd  
/home/yaroslav/project  
yaroslav@laptop:~/project$ ls  
README.md input.inp run.sh scripts src
```

Цветная часть до \ \$ — приглашение shell: она сообщает, кто и где работает. Пользователь вводит только текст **после** \ \$.

Файловая система — одно дерево



В Linux всё начинается от корня `/`.

- `/home` — домашние каталоги пользователей
- `/etc` — системные настройки
- `/usr` — большинство программ
- `/var` — изменяемые данные и логи
- `/tmp` — временные файлы

Пути: абсолютные и относительные

Абсолютный путь

Начинается с `/`.

```
/home/yaroslav/projects/  
analysis.py
```

- `~` — домашний каталог пользователя;
- `.` — текущий каталог;
- `..` — каталог уровнем выше.

Относительный путь

Считается от текущей папки.

```
scripts/run.sh
```

Где я нахожусь?

```
yaroslav@laptop:~/project$ pwd
/home/yaroslav/project
yaroslav@laptop:~/project$ ls -ld scripts
drwxr-xr-x 2 yaroslav students 4096 Mar 20 scripts
yaroslav@laptop:~/project$ cd scripts
yaroslav@laptop:~/project/scripts$ cd ..
yaroslav@laptop:~/project$ cd ~
yaroslav@laptop:~$ pwd
/home/yaroslav
```

`pwd` — где мы; `ls -ld scripts` — сведения о папке; `cd` — переход между папками.

Создаём, копируем, перемещаем

```
yaroslav@laptop:~/project$ mkdir results
yaroslav@laptop:~/project$ touch notes.txt
yaroslav@laptop:~/project$ cp input.inp input.backup
yaroslav@laptop:~/project$ mv notes.txt README-notes.txt
yaroslav@laptop:~/project$ mkdir empty-directory
yaroslav@laptop:~/project$ rmdir empty-directory
```

`mkdir` создаёт каталог, `touch` — пустой файл, `cp` копирует, `mv` перемещает или переименовывает.

Смотрим содержимое и место на диске

Текст и тип файла

`cat file.txt` — короткий файл
`less output.log` — листать большой файл
`head -n 20 file` / `tail -n 20 file`
`file trajectory.dump`

Размеры и программы

`du -sh results/` — размер каталога
`df -h` — свободное место
`which python` — исполняемый файл в PATH
`whereis lammps` — связанные файлы
`clear` — очистить экран

Осторожно: удаления часто необратимы

НЕ ЗАПУСКАТЬ

Опасный пример: никогда не вводите эту строку

```
sudo rm -rf /
```

Команда пытается удалить всё дерево файловой системы.

Перед удалением

`rm file` удаляет файл; `rm -r directory` — каталог с содержимым.

Проверьте путь через `pwd` и `ls`. `rm -rf` не спрашивает подтверждения.

Wildcard *: полезно, но опасно

```
yaroslav@laptop:~/project$ ls *.tmp  
cache.tmp failed-run.tmp notes.tmp  
yaroslav@laptop:~/project$ rm *.tmp
```

* разворачивается shell в список подходящих имён **до запуска команды**. Сначала показывайте совпадения через `ls`, затем удаляйте только то, что действительно нужно.

Права доступа

```
yaroslav@laptop:~/project$ ls -l run.sh  
-rw-r--r-- 1 yaroslav students 428 Mar 20 10:18 run.sh
```

Владелец

первые три символа

Группа

следующие три

Остальные

последние три

r — читать, w — менять, x — запускать / заходить в каталог.

chmod и chown

```
yaroslav@laptop:~/project$ chmod +x run.sh
yaroslav@laptop:~/project$ chmod 755 run.sh
yaroslav@laptop:~/project$ ls -l run.sh
-rwxr-xr-x 1 yaroslav students 428 Mar 20 10:18 run.sh
```

Число **755** читается тройками: $7 = rwx$, $5 = r-x$, $5 = r-x$.

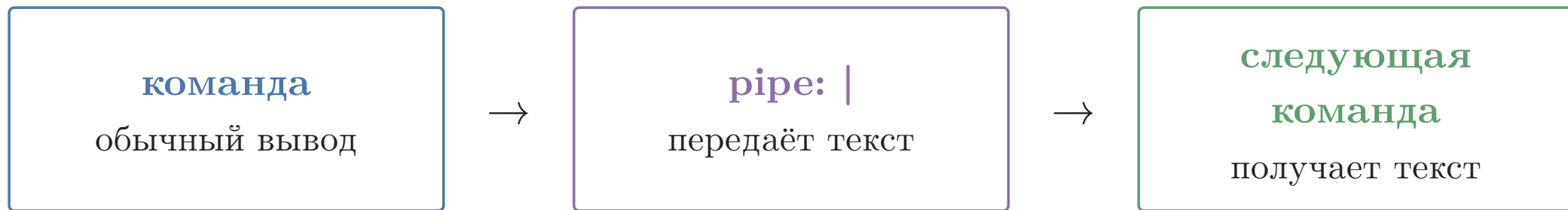
chown user:group file меняет владельца; обычно это делает администратор или владелец с нужными правами.

sudo: команда с правами администратора

```
yaroslav@laptop:~/project$ apt update
E: Could not open lock file ... Permission denied
yaroslav@laptop:~/project$ sudo apt update
Пакетные списки обновлены
```

`sudo command` временно запускает **одну** команду от имени администратора. Не работайте постоянно под `root`: ошибка с правами `root` обходится системе гораздо дороже.

Потоки и pipe



У процесса есть ввод, обычный вывод и сообщения об ошибках. Shell умеет соединять их без промежуточных файлов.

Перенаправление и pipeline

```
yaroslav@laptop:~/project$ ls -la | less
# закрываем less клавишей q
yaroslav@laptop:~/project$ ps aux | grep python
yaroslav@laptop:~/project$ python analysis.py > output.txt
yaroslav@laptop:~/project$ python analysis.py >> run.log
```

> создаёт или перезаписывает файл, >> дописывает в конец, | передаёт stdout другой программе. Ошибки можно отдельно направить через 2> errors.log.

Поиск: grep и find

```
yaroslav@laptop:~/project$ find . -name "*.py"  
./src/analysis.py  
./scripts/plot.py  
yaroslav@laptop:~/project$ grep -R "TODO" .  
./src/analysis.py:# TODO: validate units
```

`find` ищет по именам и условиям; `grep` ищет текст внутри файлов.

Маленькие инструменты, большой результат

```
yaroslav@laptop:~$ cut -d, -f2 data.csv | sort | uniq | wc -l  
12
```

В примере `data.csv` лежит в домашнем каталоге и содержит измерения: 12 разных значений во втором столбце.

- `sort` сортирует строки;
- `uniq` объединяет соседние одинаковые строки;
- `wc -l` считает строки;
- `cut` выделяет столбцы простого текстового формата.

Процессы: что сейчас выполняется

```
yaroslav@laptop:~/project$ ps aux | grep python
yaroslav 4182 98.4 python simulation.py
yaroslav@laptop:~/project$ top
PID USER %CPU COMMAND
# q — выйти из top
```

Процесс — запущенная программа с PID. `ps` показывает снимок, `top` обновляет его; `htop` часто удобнее, если установлен.

Работающая программа: на экране и в фоне

```
yaroslav@laptop:~/project$ python simulation.py &  
[1] 4182  
yaroslav@laptop:~/project$ jobs  
[1]+  Running python simulation.py &  
yaroslav@laptop:~/project$ fg %1  
# Ctrl+C — остановить расчёт, прежде чем вводить новые команды
```

`Ctrl`+`C` прерывает программу на экране. `Ctrl`+`Z` приостанавливает её; затем `bg` продолжает в фоне, `fg` возвращает. На удалённом сервере это не заменяет `tmux`.

Останавливать процесс — по нарастающей

```
yaroslav@laptop:~/project$ kill 4182  
# просим процесс корректно завершиться  
# Только если процесс НЕ завершился: kill -9 4182
```

Сначала сохраните данные и попробуйте `Ctrl+C` или `kill PID`. `kill -9` нельзя перехватить: процесс не успеет убрать временные файлы.

Установка программ: разные уровни

Системный менеджер

Ubuntu: `sudo apt update` и `sudo apt install tmux`. Ставит программы для ОС.

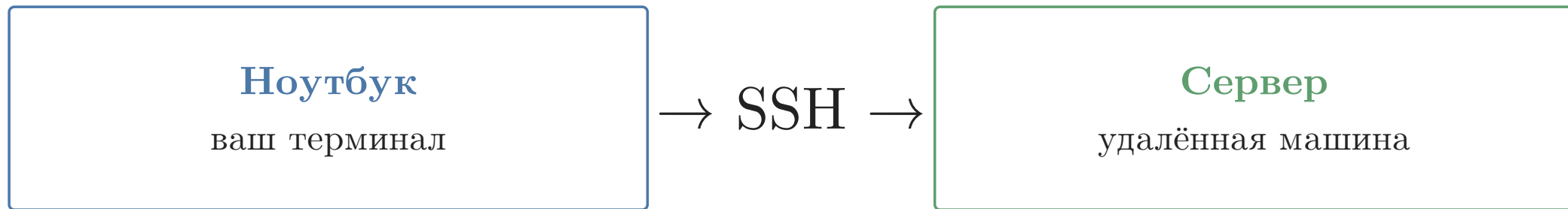
Менеджер языка

`pip`, `uv`, `cargo` управляют библиотеками и инструментами конкретной экосистемы.

Один и тот же пакет не всегда следует искать на обоих уровнях: сначала поймите, что именно устанавливаете.

SSH: работаем на другой машине

Что делает SSH



SSH открывает shell на удалённой машине через сеть. Вы видите терминал, но команды, файлы и процессы находятся на сервере.

Самое простое подключение

```
yaroslav@laptop:~/project$ ssh yaroslav@192.168.1.50  
yaroslav@server:~$ exit
```

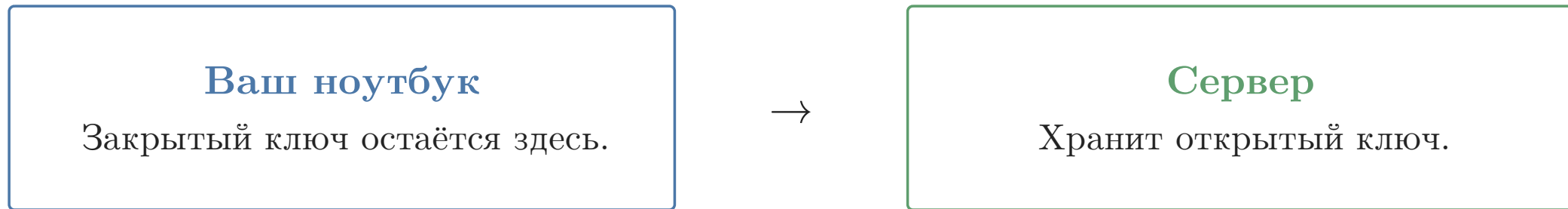
`yaroslav` — пользователь на **удалённой** машине; `@` отделяет его от адреса. `exit` завершает удалённую сессию и возвращает вас на ноутбук.

Первое подключение: проверяем личность сервера

```
yaroslav@laptop:~/project$ ssh yaroslav@192.168.1.50
The authenticity of host '192.168.1.50' can't be established.
ED25519 key fingerprint is SHA256:...
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Fingerprint нужно сверить с надёжным источником: документацией кластера или администратором. После подтверждения ключ сервера сохраняется в `~/.ssh/known_hosts`.

SSH-ключ: что хранится на каждой машине



Закрытый ключ никуда не отправляется. Сервер проверяет, что он есть у вас; открытый ключ лежит в `~/.ssh/authorized_keys` на сервере.

Создаём ключ ED25519

```
yaroslav@laptop:~/project$ ssh-keygen -t ed25519
Your identification has been saved in /home/yaroslav/.ssh/id_ed25519
Your public key has been saved in /home/yaroslav/.ssh/id_ed25519.pub
yaroslav@laptop:~/project$ chmod 700 ~/.ssh
yaroslav@laptop:~/project$ chmod 600 ~/.ssh/id_ed25519
```

Private key — секрет: не отправляйте его почтой, в Git или на сервер.

Public key можно добавлять в authorised keys.

Добавляем public key на сервер

```
yaroslav@laptop:~/project$ ssh-copy-id yaroslav@server.example.org  
Number of key(s) added: 1  
Now try logging in with: ssh yaroslav@server.example.org
```

Команда дописывает **public key** в `~/.ssh/authorized_keys` на сервере. Если `ssh-copy-id` недоступен, public key добавляют туда вручную через защищённый канал.

~/.ssh/config: короткие и повторяемые команды

```
Host chemistry
  HostName server.example.org
  User yaroslav
  IdentityFile ~/.ssh/id_ed25519
Host cluster
  HostName cluster.example.org
  User yaroslav
  IdentityFile ~/.ssh/id_ed25519
```

SSH config в действии и диагностика

```
yaroslav@laptop:~/project$ ssh chemistry
yaroslav@server:~$ exit
yaroslav@laptop:~/project$ ssh -v chemistry
# -vv и -vvv дают ещё больше диагностической информации
# ServerAliveInterval=60 можно задать в ~/.ssh/config
```

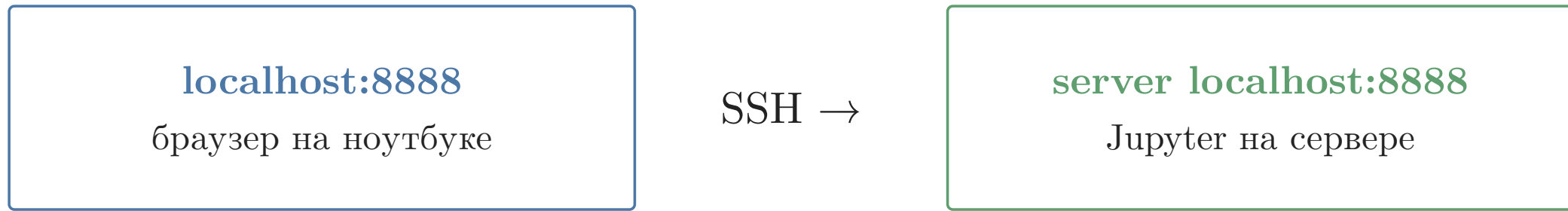
`Host` — псевдоним, `HostName` — адрес. `ssh -v` — первый инструмент при проблемах. `ServerAliveInterval` можно задать в config для keepalive; не включайте `ForwardAgent` глобально без необходимости.

Передаём файлы: scp и rsync

```
yaroslav@laptop:~/project$ scp input.dat cluster:~/project/  
yaroslav@laptop:~/project$ scp cluster:~/project/result.out .  
yaroslav@laptop:~/project$ rsync -avP ./ cluster:~/project/
```

`scp` удобен для одного файла. `rsync -avP` синхронизирует каталоги: -а сохраняет структуру, -v показывает ход, -P показывает прогресс и позволяет продолжить передачу.

Туннель SSH: доступ к сервису на сервере



```
yaroslav@laptop:~/project$ ssh -L 8888:localhost:8888 cluster
```

-L открывает **локальный** порт и передаёт его на указанный адрес с точки зрения сервера. `ssh -D 1080 server` создаёт SOCKS proxy — это другой, более общий режим.

Базовая SSH-гигиена

- Используйте ключи и защищайте private key passphrase.
- Не копируйте private key на сервер и не коммитьте его.
- Не отключайте проверку host key «чтобы быстрее подключиться».
- Не входите под root без необходимости.
- Открывайте наружу только действительно нужные сервисы.

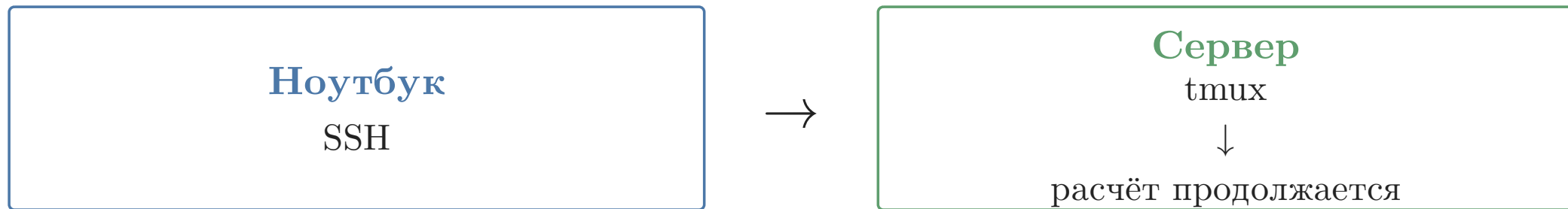
Проблема длинного расчёта

```
yaroslav@laptop:~/project$ ssh cluster  
yaroslav@cluster:~$ python long_calculation.py  
Connection to cluster closed by remote host.
```

Если Wi-Fi пропал, SSH-сессия завершается. Запущенный процесс может получить hangup и остановиться. Нужна сессия, которая живёт на сервере.

tmux

tmux: рабочее окно на сервере



tmux создаёт рабочее окно на сервере. SSH можно отключить; tmux и запущенные внутри него программы продолжают работать.

Минимальный workflow tmux

```
yaroslav@cluster:~$ tmux new -s calc
yaroslav@cluster:~$ python simulation.py
# Ctrl+b, затем d — вернуться в shell сервера
yaroslav@cluster:~$ tmux ls
calc: 1 windows (created Wed Sep 23) (detached)
yaroslav@cluster:~$ tmux attach -t calc
```

Выйдя из сессии через `Ctrl+B`, затем `D`, можно закрыть SSH. Ненужную сессию удаляют командой `tmux kill-session -t calc`.

Как читать сочетания tmux

Одновременно

`Ctrl`+`C` — удерживать Ctrl и нажать C.

Последовательно

`Ctrl+B`, затем `D` — prefix, отпустить, затем следующая клавиша.

В tmux `Ctrl`+`B` — prefix: он сообщает tmux, что следующая клавиша — команда, а не ввод в shell.

Окна tmux

Новое окно

`Ctrl+B`, затем `C`

Навигация

`Ctrl+B`, затем `N` /

`Ctrl+B`, затем `P`

По номеру

`Ctrl+B`, затем `0...9`

`Ctrl+B`, затем `,` переименовывает текущее окно. Окна полезны для независимых задач: editor, расчёт, мониторинг.

tmux: что ещё полезно знать

Разделить окно

`Ctrl+B`, затем `%` — рядом
появится второе окно.

Переключение: `Ctrl+B` + `стрелки`.

Прокрутить вывод

Сначала нажмите `Ctrl+B`, затем
клавишу левой квадратной
скобки; `q` — выйти.

Сначала достаточно освоить одну сессию и несколько окон. Разделение экрана пригодится позже для расчёта и просмотра лога.

Практический сценарий tmux

```
yaroslav@laptop:~/project$ ssh cluster
yaroslav@cluster:~$ tmux new -s chemistry
yaroslav@cluster:~$ cd ~/calculations/job01
yaroslav@cluster:~/calculations/job01$ ./run.sh
# Ctrl+b, затем d — расчёт остаётся в tmux
yaroslav@cluster:~$ exit
```

Позже: `ssh cluster` → `tmux attach -t chemistry`.

Редакторы: Nano, Vim, VS Code

Три разных подхода

Nano

Просто исправить файл в терминале.

Vim

Работа с текстом без мыши.

VS Code

Редактор с окнами и работой по SSH.

«Лучшего» редактора нет: выбирайте инструмент под задачу и среду.

Nano: открыть, сохранить, выйти

```
yaroslav@laptop:~/project$ nano input.inp  
^O Write Out ^W Where Is ^K Cut Text  
^X Exit ^U Paste Text
```

Сохранить

Ctrl + O

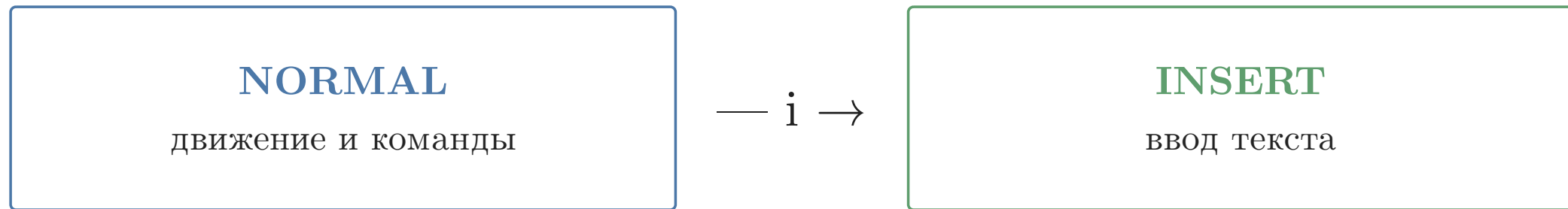
Выйти

Ctrl + X

Найти

Ctrl + W

Vim начинается не с клавиш, а с режимов



Esc возвращает в обычный режим. **v** выделяет текст, **:** открывает строку команд.

Главное в Vim — понимать, **в каком режиме вы сейчас находитесь**.

Vim: открыть → изменить → сохранить

```
yaroslav@laptop:~/project$ vim file.txt
i # перейти в INSERT
текст...
Esc # вернуться в NORMAL
:w # сохранить
:q # выйти; :wq — сохранить и выйти; :q! — выйти без сохранения
```

Vim: навигация и редактирование

Движение в NORMAL

`h j k l` — влево / вниз / вверх /

вправо

`w` / `b` — слово вперёд / назад

`0` / `$` — начало / конец строки

`gg` / `G` — начало / конец файла

Изменение

`i` / `a` / `o` — вставка

`dd` — удалить строку; `yy` —
копировать

`p` — вставить; `u` / `Ctrl+r` —
undo / redo

`x` — удалить символ

Vim: короткие команды складываются в действия

```
d — удалить  
w — следующее слово  
dw — удалить слово  
cw — заменить слово
```

Вместо заучивания сотен сочетаний полезно видеть логику: первая клавиша задаёт действие, следующая — к чему оно относится.

Vim: поиск и выделение

Поиск и замена

`/text` — искать; `n / N` —
следующее / предыдущее;
`:%s/old/new/g` — заменить во
всём файле.

Выделение

`v` — посимвольное, `V` —
построчное. Затем `y` копирует, `d`
удаляет.

VS Code: редактор с графическим интерфейсом

`Ctrl` + `P` — открыть файл

`Ctrl` + `Shift` + `P` — палитра

команд

`Ctrl` + `Shift` + `F` — поиск по

проекту

`Ctrl` + `grave` — встроенный

терминал

`Ctrl` + `B` — боковая панель

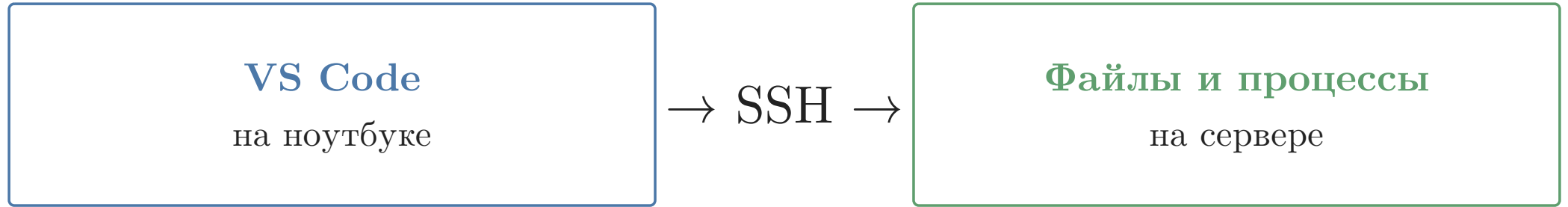
`Ctrl` + `Space` — подсказки

`F2` — переименовать символ

`Ctrl` + `S` — сохранить

На macOS часто заменяют `Ctrl` на `Cmd`; сверяйтесь с настройками `keybindings`.

VS Code Remote SSH



Расширение **Remote — SSH** берёт адрес из `~/.ssh/config`. В палитре команд выберите **Remote-SSH: Connect to Host...** → `cluster`. Окно VS Code останется на ноутбуке, а файлы откроются с сервера.

Git: история и совместная работа

Что такое Git

Git — распределённая система контроля версий. Она сохраняет историю проекта, показывает изменения, позволяет экспериментировать в ветках и обмениваться работой.

Подходит

код, input files, скрипты, README, конфигурация

Не заменяет

резервное копирование больших бинарных результатов

Git $\neq$ GitHub

Git

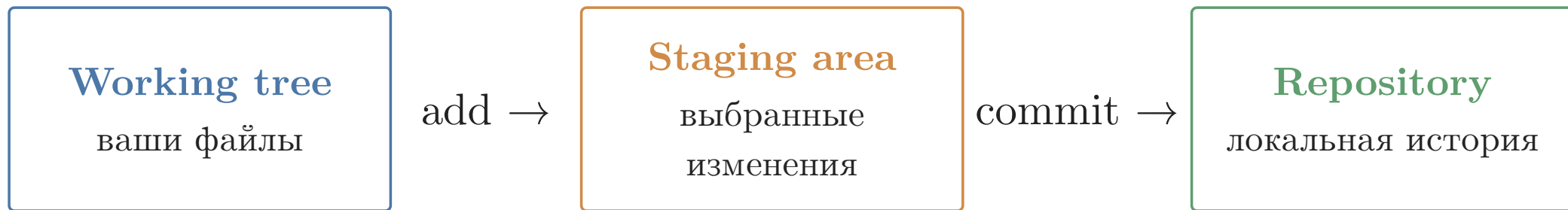
технология и программа контроля версий

GitHub

один из сервисов для Git-репозиторий и
совместной работы

Есть и другие сервисы: GitLab, Gitea, университетский сервер; Git может работать полностью локально.

Три зоны Git



`git add` выбирает изменения, `git commit` сохраняет их в истории. Так можно собрать один осмысленный commit.

Создать репозиторий и посмотреть статус

```
yaroslav@laptop:~$ mkdir project
yaroslav@laptop:~$ cd project
yaroslav@laptop:~/project$ git init -b main
Initialized empty Git repository
yaroslav@laptop:~/project$ git status
On branch main
No commits yet
```

`git status` — полезная привычка: запускайте её до и после действий.

`git init -b main` явно задаёт имя первой ветки.

Первый commit

```
yaroslav@laptop:~/project$ printf 'print(42)\n' > main.py
yaroslav@laptop:~/project$ git status
Untracked files: main.py
yaroslav@laptop:~/project$ git add main.py
yaroslav@laptop:~/project$ git status
Changes to be committed: new file: main.py
yaroslav@laptop:~/project$ git commit -m "Add initial script"
```

Commit — небольшая завершённая мысль с понятным сообщением, а не случайный снимок всего каталога.

git diff: увидеть изменения до commit

```
yaroslav@laptop:~/project$ printf 'print(43)\n' > main.py
yaroslav@laptop:~/project$ git diff
@@ -1 +1 @@
-print(42)
+print(43)
yaroslav@laptop:~/project$ git diff --staged
# нет вывода: изменение ещё не добавлено через git add
yaroslav@laptop:~/project$ git restore main.py
```

`git diff` показывает правки в файле. `git diff --staged` показывает изменения после `git add`. `git restore` отменяет пробное изменение перед следующим шагом.

История Git

```
yaroslav@laptop:~/project$ git log --oneline --graph --decorate --all  
* a1d7e0c (HEAD -> main) Add initial script
```

История — это не только архив: она помогает найти источник изменения и вернуться к рабочей версии.

Ветки: отдельная линия работы

```
A  main
  \
   B  experiment
```

```
yaroslav@laptop:~/project$ git switch -c experiment
yaroslav@laptop:~/project$ printf 'print(43)\n' > main.py
yaroslav@laptop:~/project$ git add main.py
yaroslav@laptop:~/project$ git commit -m "Try new value"
```

Современная команда `git switch` яснее для переходов между ветками.
`git checkout` часто встретится в старых инструкциях.

Merge: соединяем работу

```
yaroslav@laptop:~/project$ git switch main  
yaroslav@laptop:~/project$ git merge experiment
```

до: A main
 \
 B experiment

после: A — B main

Иногда Git делает fast-forward, иногда создаёт merge commit. Важнее понимать намерение: переносим проверенную работу из experiment в main.

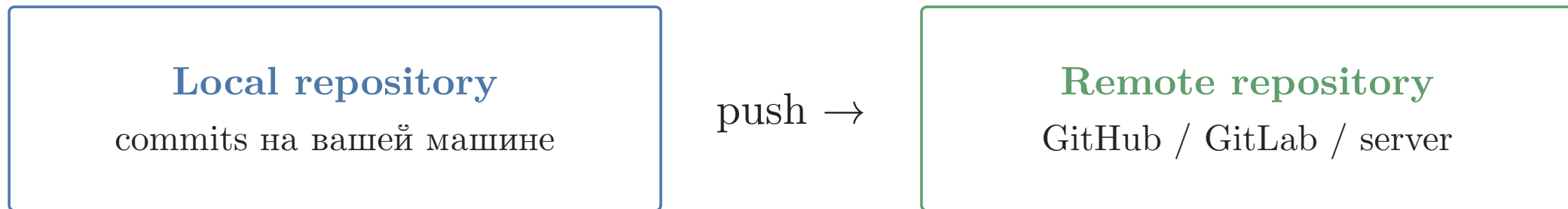
Merge conflict — это вопрос к человеку

```
<<<<<< HEAD
temperature = 300
=====
temperature = 350
>>>>>> experiment
```

1. Выберите нужное содержимое и удалите строки с маркерами конфликта.
2. Запишите решение: `git add file`.
3. Завершите слияние: `git commit`.

Git не может угадать правильный вариант за вас.

Remote: локальная и удалённая история



```
yaroslav@laptop:~/project$ git remote -v
# пока вывода нет: удалённый репозиторий не настроен
yaroslav@laptop:~/project$ git remote add origin git@github.com:u/p.git
yaroslav@laptop:~/project$ git push -u origin main
```

В примере `git@github.com:u/p.git` — адрес вашего репозитория: замените его на настоящий перед выполнением команды.

clone, fetch, pull, push

```
yaroslav@laptop:~$ git clone git@github.com:u/p.git demo
yaroslav@laptop:~$ cd demo
yaroslav@laptop:~/demo$ git fetch
# скачать новую историю, не меняя working tree
yaroslav@laptop:~/demo$ git pull
# fetch + интеграция изменений
yaroslav@laptop:~/demo$ git push
```

SSH-адрес в `git clone` использует те же SSH keys, что и вход на сервер.

.gitignore: что не должно попадать в историю

```
.venv/  
__pycache__/  
*.pyc  
results/  
output.log  
large_trajectory.dump
```

.gitignore не удаляет файл, если он уже tracked. Тогда сначала убирают его из индекса осознанной командой `git rm --cached file`.

Git-гигиена для научного проекта

Не коммитить

- private SSH keys, пароли, API tokens, `.env` с секретами;
- виртуальное окружение и build artifacts;
- огромные trajectories без ясной причины.

Обычно коммитить

- input files, scripts, analysis code;
- README, конфигурацию, зависимости;
- маленькие тестовые данные.

Для больших данных нужны отдельное хранилище, архив или Git LFS при необходимости.

Структура научного Git-проекта

```
lammps_rutin_mg/
├── runs/
│   ├── ferrum_solvation/           # Папка расчёта, со всеми зависимостями
│   │   ├── in.properties.lmp      # Скрипт для запуска LAMMPS-расчёта
│   │   └── run_properties.sh       # Скрипт для запуска серии расчётов
│   └── analysis/
│       ├── ferrum_solvation/       # Папка с анализом результатов расчёта
│       │   ├── solvation.typ
│       │   └── analyze.py
│       └── literature/             # Папка с литературным обзором по расчёту
│           └── main.typ
├── README.md                       # Файл-описание дирекории
└── .gitignore
```

`runs/` — расчёты, `analysis/` — обработка, `literature/` — текст работы.

Практический Git-цикл

```
yaroslav@laptop:~/project$ git pull
yaroslav@laptop:~/project$ git status
# работа
yaroslav@laptop:~/project$ git diff
yaroslav@laptop:~/project$ git add analysis.py
yaroslav@laptop:~/project$ git commit -m "Add RDF analysis"
yaroslav@laptop:~/project$ git push
```

Осторожно с git add .

Команда удобна, но сначала посмотрите `git status` и `git diff`: в staging area не должны случайно попасть результаты, секреты или чужие файлы.

Один связный workflow

От ноутбука к расчёту и обратно

```
yaroslav@laptop:~/project$ ssh chemistry
yaroslav@server:~$ git clone git@github.com:group/project.git
yaroslav@server:~$ cd project
yaroslav@server:~/project$ tmux new -s calc
yaroslav@server:~/project$ vim input/calculation.inp
yaroslav@server:~/project$ bash run.sh
# Ctrl+b, затем d — расчёт остаётся на сервере
yaroslav@server:~/project$ exit
```

Проверяем результат и сохраняем историю

```
yaroslav@laptop:~/project$ ssh chemistry
yaroslav@server:~$ tmux attach -t calc
# расчёт завершился; Ctrl+b, затем d – вернуться в shell
yaroslav@server:~$ cd project
yaroslav@server:~/project$ tail -n 20 output.log
# после редактирования scripts/analysis.py
yaroslav@server:~/project$ git status
yaroslav@server:~/project$ git diff
yaroslav@server:~/project$ git add scripts/analysis.py
yaroslav@server:~/project$ git commit -m "Improve analysis"
yaroslav@server:~/project$ git push
```

VS Code Remote SSH может заменить Vim для редактирования, но базовая модель остаётся той же: файлы и процессы находятся на сервере.

Шпаргалка: Linux и SSH

Linux

`pwd` · `ls` · `cd`

`mkdir` · `cp` · `mv`

`less` · `grep` · `find`

`ps` · `tail -f`

SSH

`ssh host`

`ssh-keygen -t ed25519`

`ssh-copy-id user@host`

`scp` · `rsync -avP`

`ssh -L 8888:localhost:8888 host`

Шпаргалка: tmux, Vim и Git

tmux

`new -s name`

`attach -t name`

`Ctrl+B`, затем `D`

`Ctrl+B`, затем `C`

`Ctrl+B`, затем `%`

Vim

`i` · `Esc`

`:w` · `:q!`

`dd` · `yy` · `p`

`/text` · `n`

Git

`status` · `diff`

`add` · `commit`

`log` · `switch`

`pull` · `push`

Минимальный рабочий набор

Linux shell → SSH → tmux → Редактор → Git

Этого набора достаточно, чтобы спокойно работать на Linux-сервере.